

	PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINHAS GERAIS Instituto de Ciências Exatas e Informática (ICEI) Departamento de Sistemas de Informação e Engenharia de Software Graduação em Engenharia de Software RELATÓRIO TÉCNICO – MÉTODO ATAM <i>(Architecture Tradeoff Analysis Method)</i>	
	PROFESSOR: CLEITON SILVA TAVARES · CRISTIANO DE MACÊDO NETO · JOÃO PAULO CARNEIRO ARAMUNI	
	DISCIPLINA: TRABALHO INTERDISCIPLINAR: APLICAÇÕES DISTRIBUÍDAS PERÍODO: 5º	
	PROJETO: Biblioo	DATA: 19/06/2026

1. Resumo Executivo

O Biblioo é uma plataforma de comunidade literária web e mobile, voltada a leitores ativos brasileiros entre 18 e 35 anos, que integra seis trilhas de recomendação personalizadas (combinando Redis, Neo4j e MySQL), comunidades com chat em tempo real via WebSocket/STOMP, identidade literária visual (DNA Literário) e o assistente conversacional Bibo, sobre uma arquitetura backend em Spring Boot (monólito modular hexagonal) com mensageria assíncrona via RabbitMQ e Outbox Pattern, implantada em Google Cloud Run com TiDB Cloud, Upstash Redis, CloudAMQP e Neo4j Aura.

A avaliação ATAM foi motivada pela necessidade de validar, antes da entrega final do Sprint 6, se a arquitetura suporta os principais diferenciais competitivos do produto frente a concorrentes como Skoob e Goodreads — em especial a entrega em tempo real de recomendações multi-datastore, a confiabilidade do chat em ambiente multi-instância serverless e a resiliência a falhas de instância sem perda de eventos de domínio.

Os atributos de qualidade priorizados foram Desempenho/Escalabilidade (latência sob carga concorrente em feed, busca, recomendações e operações sociais), Confiabilidade e Disponibilidade (entrega de mensagens de chat cross-instância e recuperação de eventos após falha de instância Cloud Run) e Segurança (autenticação JWT, autorização por papel em comunidades e gestão de secrets). A compatibilidade entre plataformas web e mobile foi avaliada qualitativamente como atributo complementar.

A análise foi sustentada por evidências objetivas: uma suíte de 71 testes de carga k6 (stress, spike, load e concorrência), com 100% de aprovação e zero falhas sistêmicas (5xx), além de verificação estrutural do código para os atributos qualitativos. As conclusões indicam uma arquitetura madura para o estágio atual do produto, com pontos fortes claros — Outbox Pattern garantindo entrega confiável de eventos, FanoutExchange entregando 100% das mensagens de chat sem perda em cenários multi-instância, e filtro JWT sem vetor de bypass — mas também riscos relevantes que dependem de planos gratuitos de infraestrutura (Neo4j Aura Free, CloudAMQP Little Lemur) e de mecanismos sem redundância de persistência.

Os riscos mais críticos identificados foram a possível saturação do pool de conexões JDBC compartilhado entre threads HTTP e consumers RabbitMQ (R-01), o esgotamento de capacidade do Neo4j Aura no tier gratuito conforme o grafo social cresce (R-03) e a perda silenciosa do estado de aprendizado da trilha de recomendação Thompson Sampling em caso de flush do Redis (R-04). As recomendações-chave incluem calibrar o `maximumPoolSize` do HikariCP em função da concorrência real entre threads HTTP e consumers, monitorar proativamente o uso de conexões do CloudAMQP e do Neo4j Aura com upgrade planejado de plano, e introduzir uma cópia persistente periódica dos parâmetros α/β do Thompson Sampling no MySQL para eliminar o ponto único de perda de aprendizado.

2. Objetivos de Negócio e Direcionadores Arquiteturais

O Biblío nasce para resolver três dores centrais do leitor brasileiro identificadas nas personas do produto (Júlia, leitora universitária, e Rafael, jovem profissional): a perda de motivação ao finalizar leituras por falta de recomendações relevantes, a dispersão de conversas literárias entre redes sociais genéricas e grupos de mensagens, e a ausência de uma identidade de leitor reconhecível dentro das plataformas atuais (Skoob, Goodreads). Para se diferenciar dessas plataformas — limitadas a registro de leituras com recomendações genéricas e interação social superficial — o negócio exige que a arquitetura sustente simultaneamente seis algoritmos de recomendação independentes, comunidades com chat em tempo real e votação democrática, identidade literária visual compartilhável e um assistente conversacional capaz de operar a plataforma por linguagem natural, sem comprometer a simplicidade e fluidez prometidas ao usuário final.

Esses objetivos estratégicos se traduzem em restrições arquiteturais concretas já adotadas no projeto: arquitetura Hexagonal em monólito modular com 11 domínios independentes (preparando o terreno para extração futura em microserviços sem reescrita), persistência poliglota (MySQL/TiDB como fonte única da verdade, Neo4j para o grafo social das recomendações, Redis como cache derivado), comunicação assíncrona via Outbox Pattern + RabbitMQ entre módulos, e um aplicativo mobile offline-first que precisa funcionar plenamente sem conexão à internet — um requisito de negócio direto para reter o leitor brasileiro em contextos de conectividade instável.

Principais direcionadores de qualidade identificados:

- **Direcionador 1 — Desempenho sob concorrência multi-datastore:** o motor de seis trilhas de recomendação, o feed social e o chat em tempo real precisam responder dentro dos limites de p(95) definidos pelos RNF-05 a RNF-19, mesmo quando cada requisição percorre simultaneamente até três datastores distintos (Redis, Neo4j e MySQL) — a engagem do produto depende de respostas rápidas mesmo nas funcionalidades mais caras computacionalmente.
- **Direcionador 2 — Confiabilidade e disponibilidade em ambiente serverless multi-instância:** o Biblio opera em Google Cloud Run com múltiplas instâncias e *scale-to-zero* no ambiente portfolio. O negócio exige que nenhuma mensagem de chat se perca entre instâncias e que nenhum evento de domínio (post, avaliação, atualização de estante) seja perdido em caso de falha abrupta de instância — daí a adoção obrigatória do Outbox Pattern e do FanoutExchange para reentrega cross-instância.
- **Direcionador 3 — Segurança e proteção de dados do usuário:** com posts, reviews, conversas e perfis privados de leitores, a arquitetura precisa garantir autenticação stateless via JWT em todos os endpoints, autorização por papel dentro de comunidades, sanitização de conteúdo gerado por usuário contra XSS e gestão de credenciais exclusivamente via Google Secret Manager — sem isso, a confiança do leitor na plataforma social fica comprometida.
- **Direcionador 4 — Modificabilidade e evolução independente de módulos:** como o roadmap do produto prevê expansão contínua de trilhas de recomendação, tipos de comunidade e canais de notificação, a arquitetura impõe que os 11 módulos de domínio nunca se conheçam diretamente, comunicando-se apenas por eventos assíncronos ou interfaces — permitindo adicionar funcionalidades sem reescrever módulos existentes nem arriscar regressões em cascata.
- **Direcionador 5 — Compatibilidade e continuidade offline no mobile:** como parte relevante da base de usuários acessa o Biblio via aplicativo Android/iOS, o negócio exige que estantes, feed, comunidades e recomendações permaneçam navegáveis sem conexão, com sincronização automática ao restabelecer a rede — refletido na escolha do padrão offline-first com Drift/SQLite e Hive.

3. Topologia e Abordagem da Arquitetura

O Biblio adota uma arquitetura **Hexagonal (Ports & Adapters)** organizada como **monólito modular** no back-end, com 11 módulos de domínio (user, books, community, dna, feed, notification, recommendation, share, trending, assistant, infrastructure) que nunca se importam diretamente entre si — toda comunicação entre módulos ocorre por eventos assíncronos via RabbitMQ ou por interfaces explícitas (ports). O sistema é poliglota em persistência: o MySQL/TiDB Cloud é a única fonte da verdade transacional, enquanto Neo4j (grafo social), Redis (cache efêmero) e OpenSearch (índice de busca)

são sempre derivados e reconstruíveis a partir dele. O backend é implantado como contêiner Docker no Google Cloud Run em dois ambientes (biblioo-portfolio, escalável a zero, e biblioo-producao, sempre ativo), com session-affinity habilitado para preservar conexões WebSocket/STOMP por instância e o RabbitMQ FanoutExchange garantindo a reentrega de mensagens entre instâncias distintas.

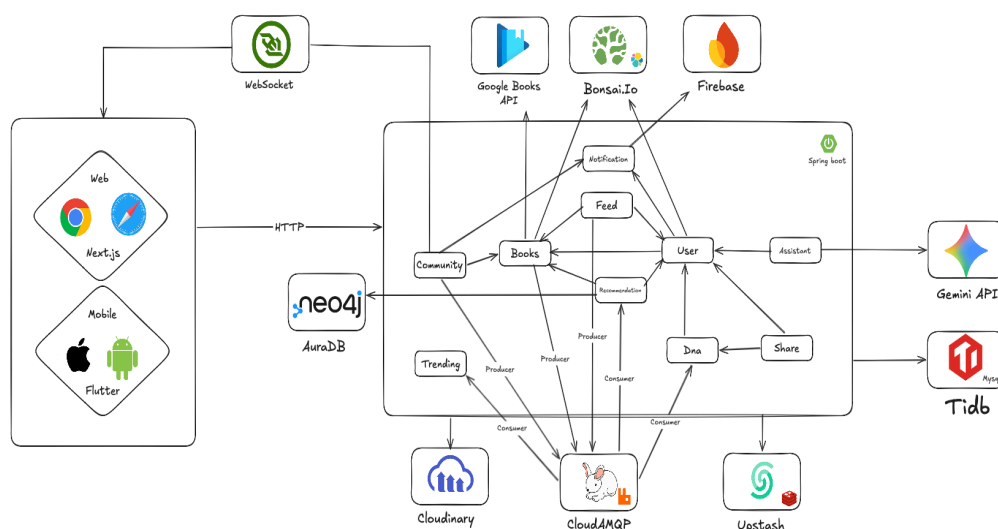


Figura 1 — Arquitetura de Domínio do Biblioo. Fonte: os próprios autores.

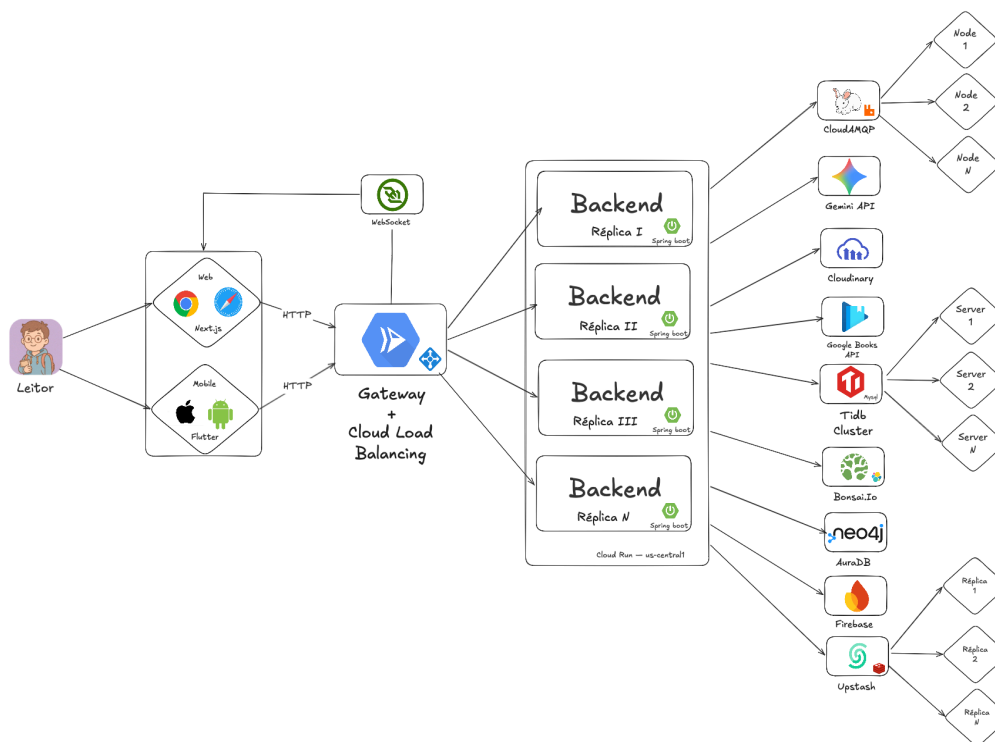


Figura 2 — Arquitetura infra do Biblío. Fonte: os próprios autores.

Componente / Camada	Tecnologia Proposta	Papel Estrutural e Mecanismo de Comunicação
Camada de Apresentação Web	Next.js 16 + React 19 + TypeScript	SPA com App Router e renderização híbrida SSR/CSR; consome a API via camada de serviços (src/services/) e abre canal WebSocket/STOMP para o chat e SSE para notificações em tempo real.
Camada de Apresentação Mobile	Flutter 3.11 + Dart 3.11 (BLoC)	App offline-first: Repository sempre tenta o datasource local (Drift/SQLite + Hive) antes do remoto; sincroniza com a API quando a rede retorna. Tokens JWT armazenados em FlutterSecureStorage.
Borda e Roteamento	Google Cloud Run (us-central1)	Hospeda os contêineres Docker do backend; gerencia escalonamento de instâncias (0–2 no portfólio, 1–10 em produção) e mantém session-affinity para conexões WebSocket.

Back-End Central	Java 25 + Spring Boot 4 (Hexagonal, monólito modular)	Motor de regras de negócio dividido em 11 módulos de domínio isolados, comunicando-se exclusivamente por eventos RabbitMQ ou interfaces — nunca por chamada direta entre módulos.
Persistência Relacional	MySQL 8.4 (local) / TiDB Cloud Serverless (produção) via Spring Data JPA + Hibernate	Fonte única da verdade de todas as entidades do domínio; nenhum outro armazenamento assume esse papel.
Grafo de Relacionamentos	Neo4j 5.18 (local) / Neo4j Aura Free (produção) via driver Java nativo	Sustenta as trilhas de recomendação por co-leitura (T1) e filtragem colaborativa (T5), acessado via Cypher direto, sem ORM de grafo.
Cache Distribuído	Redis 7.4 (local) / Upstash (produção)	Cache-Aside por domínio (feed, trending, cápsula visual) e estado efêmero (parâmetros bayesianos do Thompson Sampling, histórico do assistente); sempre derivado e reconstruível do MySQL.
Índice de Busca	OpenSearch 2.18 (local) / Bonsai.io Hobby (portfolio) / GCE VM via VPC (produção)	Índice derivado e reconstruível para busca de livros/usuários; fallback automático para FULLTEXT no MySQL em caso de falha, sem erro 5xx ao cliente.
Mensageria Assíncrona	RabbitMQ 4.0 (local) / CloudAMQP Little Lemur (produção)	14 filas sobre TopicExchange biblioo.events (eventos de domínio) e FanoutExchange biblioo.community.broadcast (chat cross-instância), com Outbox Pattern, idempotência por event_id e DLQ com retry exponencial.
Chat em Tempo Real	Spring WebSocket + STOMP + RabbitMQ FanoutExchange	Full-duplex sobre WebSocket; CommunityBroadcastConsumer replica mensagens para todas as instâncias Cloud Run ativas, filtrando por INSTANCE_ID para evitar duplicatas.

Notificações	SSE (web) via Spring SseEmitter / Firebase Admin SDK (FCM, mobile)	Push unidirecional servidor → cliente sem polling (web) e entrega assíncrona a dispositivos Android/iOS (mobile).
Autenticação e Autorização	Spring Security + JJWT 0.12.6 + Google OAuth 2.0	Sessão stateless via JWT (Access Token 1h, Refresh Token 7 dias com rotação); JwtAuthenticationFilter intercepta toda requisição antes dos controllers; rate limiting via Bucket4j antes do pool JDBC.
Assistente Conversacional (Bibo)	Spring AI 2.0.0-M5 + Google Gemini GenAI API	LLM com function calling para executar ações na plataforma (criar estantes, atualizar progresso, sugerir leituras) a partir de linguagem natural, com histórico de conversa em cache.
Armazenamento de Mídia	Cloudinary HTTP5 2.0.0 + Apache Tika 3.2.3	Upload assíncrono de avatares, banners e imagens de posts, com validação de MIME type real antes do processamento.
Catálogo Externo de Livros	Google Books API (HTTP/REST)	Fonte de fallback para expandir o catálogo; livros retornados são persistidos no MySQL para resolução interna em buscas futuras, sem erro 5xx ao cliente em caso de falha externa.
E-mail Transacional	SendGrid API (Spring Mail/SMTP como fallback)	Envio de e-mails de redefinição de senha — único tipo de comunicação por e-mail enviado pela plataforma.
Deploy e CI/CD	Docker + Google Artifact Registry + Cloud Build + GitHub Actions	Pipeline acionado por push na branch prod, com build e deploy em ~12 minutos sem downtime; secrets gerenciados exclusivamente via Google Secret Manager (40+ secrets).
Observabilidade	Micrometer + Prometheus + Grafana	Coleta de métricas HTTP, pool de conexões JDBC e desempenho de algoritmos de recomendação.

4. Árvore de Utilidade e Inventário de Cenários

A árvore de utilidade do Biblioio priorizou os atributos de qualidade diretamente ligados aos diferenciais competitivos do produto — motor de recomendação multi-datastore, comunidade em tempo real e confiabilidade de eventos em ambiente serverless — em detrimento de atributos secundários como compatibilidade de plataforma, já cobertos por testes manuais qualitativos.

ID	Atributo de Qualidade	Descrição do Cenário (Estímulo, Ambiente e Resposta)	Prioridade (Negócio / Dificuldade)
C-01	Desempenho / Escalabilidade	600 usuários acessam o feed social, publicam posts/comentários, registram avaliações e atualizam estantes simultaneamente por 2 minutos (pico de uso), com feed pré-carregado em cache Redis e fanout assíncrono via RabbitMQ. O sistema deve carregar o feed em até 5.000ms (p95), confirmar posts/comentários em até 1.500ms, registrar avaliações em até 5.000ms, exibir perfil em até 8.000ms e confirmar atualização de estante em até 2.000ms — sem falhas 5xx e erro < 1%.	Alta / Alta
C-02	Desempenho / Escalabilidade	Simultaneamente: 100 usuários buscam livros, 400 consultam as 6 trilhas de recomendação (cada requisição percorrendo Redis, Neo4j e MySQL), 800 acionam o Roll Dice, 300 acessam livros em tendência e 200 geram seu DNA Literário/cápsula visual. O sistema deve responder dentro dos limites de p95 declarados por funcionalidade (4.000ms a 10.000ms), mantendo a integridade dos resultados por usuário, sem cruzamento de dados entre sessões.	Alta / Alta
C-03	Confiabilidade	250 usuários trocam mensagens via WebSocket/STOMP em múltiplas salas, distribuídos entre instâncias Cloud Run; outros 250 seguem/deixam de seguir; 600 executam operações administrativas de comunidade. O sistema deve entregar 100% das mensagens sem perda mesmo entre instâncias distintas, com p95 < 2.000ms para até 40 membros simultâneos e < 5.000ms para todos os conectados; operações sociais em até 5.000ms e administrativas em até 10.000ms.	Alta / Média

C-04	Disponibilidade	Uma instância Cloud Run é encerrada abruptamente durante pico, com transações de escrita em andamento (post, avaliação, atualização de estante). O sistema deve permanecer disponível para usuários conectados a outras instâncias, e os eventos gerados antes da falha devem ser processados após a recuperação, sem perda de dados nem intervenção manual — garantido pelo Outbox Pattern (evento gravado na mesma transação da operação de negócio).	Alta / Alta
C-05	Segurança	(a) Usuário não autenticado ou com JWT expirado tenta acessar endpoints protegidos (feed, estante, comunidade privada). (b) Membro comum tenta executar operação administrativa restrita (excluir membros, fechar votação, apagar mensagens). O sistema deve rejeitar com 401/403 antes de qualquer lógica de negócio ser executada.	Alta / Média
C-06	Compatibilidade de Plataforma	Um leitor acessa o Biblío via Chrome (v127+) e Safari (v17+) em resoluções de 1280px a 1920px, e via app mobile em Android 14+ e iOS 26.5+ em telas de 390px a 430px. Feed, estantes, comunidades, recomendações, DNA Literário e chat devem ser exibidos corretamente e com navegação funcional em todas as combinações.	Média / Baixa

C-01 e C-02 ficam em Alta/Alta porque atendem ao núcleo de engajamento e ao diferencial de recomendação multi-estratégia do produto, exigindo coordenação entre três datastores sob carga real. C-04 também é Alta/Alta pela complexidade intrínseca de garantir zero perda de eventos em ambiente serverless com escalonamento dinâmico. C-06 fica em Média/Baixa por ser tecnicamente resolvido por frameworks responsivos.

5. Inventário Analítico Rastreável (Saídas do ATAM)

5.1 Riscos Identificados (R)

R-01: Esgotamento do pool de conexões JDBC (HikariCP) compartilhado

- **Cenários vinculados:** C-01, C-02
- **Componente crítico:** HikariCP / pool de conexões MySQL-TiDB
- **Descrição técnica:** Threads HTTP e threads de consumer RabbitMQ competem pelo mesmo pool de conexões JDBC, sem reserva dedicada para cada tipo de consumidor. Sob carga assíncrona sustentada elevada — por exemplo, processamento simultâneo de recálculo de recomendações, DNA Literário e fanout de feed — o pool pode saturar de forma invisível nos logs de API, gerando

ConnectionTimeoutException silenciosa nos consumers enquanto a API HTTP aparenta operar normalmente. O risco se materializa quando o maximumPoolSize não é calibrado em função de (threads HTTP + total de threads de consumer ativos).

R-02: Capacidade do Neo4j Aura Free sob crescimento do grafo social

- **Cenários vinculados:** C-02
- **Componente crítico:** Neo4j Aura Free (trilhas T1 — Because You Read e T5 — Similar Authors)
- **Descrição técnica:** As consultas de co-leitura e filtragem colaborativa percorrem subgrafos cujo tamanho cresce proporcionalmente ao número de usuários e relações :READ na base. O tier gratuito do Neo4j Aura tem heap de memória limitado (~1 GB) e conexões concorrentes restritas, sem escalonamento automático. Já sob 400 VUs, as trilhas de recomendação dominam a latência p(95) com 1.210ms — o maior valor de toda a bateria de testes. Conforme o grafo social densifica organicamente, essas consultas podem cruzar o threshold de 5.000ms do RNF-15 de forma progressiva, sem qualquer alerta automático antes da degradação perceptível pelo usuário.

R-03: Perda silenciosa do estado de aprendizado da trilha Thompson Sampling

- **Cenários vinculados:** C-02
- **Componente crítico:** Upstash Redis (trilha T4 — CatalogSurprise)
- **Descrição técnica:** Os parâmetros bayesianos α/β do algoritmo Thompson Sampling, que sustentam a trilha de recomendação por descoberta, são armazenados exclusivamente no Redis com TTL de 90 dias, sem cópia persistente no MySQL. Um flush de cache por pressão de memória no plano gratuito do Upstash, ou uma troca de instância não gerenciada, destrói silenciosamente o histórico de aprendizado de todos os usuários — o modelo retorna ao prior uniforme Beta(1,1) sem qualquer aviso ou log de alerta, degradando a qualidade da trilha até o reaprendizado orgânico.

R-04: Janela de atraso do Outbox sob scale-to-zero

- **Cenários vinculados:** C-04
- **Componente crítico:** Scheduler do Outbox Pattern (@Scheduled) no ambiente bibliao-portfolio
- **Descrição técnica:** O ambiente portfolio opera com min-instances=0, escalando a zero entre requisições. O scheduler de publicação do Outbox só executa em instâncias ativas. Se uma instância é encerrada pelo Cloud Run após o commit de uma operação mas antes do disparo do scheduler, o evento permanece com status PENDING na tabela outbox_events. Não há perda permanente — o próximo startup processa os pendentes —, mas a latência de entrega pode estender-se de

segundos a minutos em baixo tráfego, o que se torna relevante caso o ambiente seja promovido a produção sem ajuste do min-instances.

R-05: Janela de acesso residual em WebSocket após revogação de membership

- **Cenários vinculados:** C-03, C-05
- **Componente crítico:** Canal WebSocket/STOMP (autenticação por handshake)
- **Descrição técnica:** A autenticação STOMP é verificada exclusivamente no handshake inicial via JWT, sem revalidação contínua por mensagem. Quando um membro é removido de uma comunidade, a revogação é persistida e o evento publicado via RabbitMQ — mas a conexão STOMP preexistente do usuário removido permanece ativa até timeout ou reconexão voluntária, permitindo envio/recebimento de mensagens na sala da qual foi removido durante essa janela. Operações REST são imunes, pois @PreAuthorize consulta o banco a cada requisição; o canal WebSocket é stateful por design e não possui esse mecanismo.

R-06: Tokens FCM não vinculados ao ciclo de vida da sessão JWT

- **Cenários vinculados:** C-05
- **Componente crítico:** device_tokens / Firebase Cloud Messaging
- **Descrição técnica:** Tokens de dispositivo FCM não são revogados no logout nem na exclusão de conta — o encerramento de sessão atua apenas sobre o canal JWT, enquanto o canal de push permanece ativo de forma independente. Em dispositivos compartilhados ou transferidos, o usuário anterior pode continuar recebendo notificações com metadados de atividade do usuário atual até a expiração orgânica do token pelo Firebase, sem prazo determinístico.

R-07: Ausência de contrato versionado de API entre clientes web e mobile

- **Cenários vinculados:** C-06 (cenário de crescimento — evolução de contrato)
- **Componente crítico:** API REST compartilhada entre Next.js e Flutter
- **Descrição técnica:** A API não possui versionamento explícito, sendo consumida por um cliente web com deploy instantâneo (Next.js) e um cliente mobile sujeito a revisão das lojas (1 a 7 dias). Uma mudança de contrato no backend mantém usuários mobile na versão anterior em estado de incompatibilidade durante toda a janela de revisão, sem mecanismo de *forced-update* ou degradação graceful — risco de modificabilidade que se materializa a cada evolução de API que quebre compatibilidade retroativa.

R-08: Limite de conexões do CloudAMQP Little Lemur sob escalonamento máximo

- **Cenários vinculados:** C-03, C-04
- **Componente crítico:** CloudAMQP Little Lemur (RabbitMQ em produção)
- **Descrição técnica:** O plano gratuito permite até 20 conexões simultâneas. Com até 10 instâncias Cloud Run ativas, cada uma abrindo de 6 a 8 conexões AMQP

para consumers e publishers, o limite pode ser atingido sob escalonamento máximo, bloqueando novas conexões e comprometendo a publicação/consumo de eventos — incluindo o FanoutExchange do chat e o pipeline do Outbox.

5.2 Pontos de Não-Risco (NR)

NR-01: Outbox Pattern elimina a janela de falha entre commit e publicação

- **Cenários vinculados:** C-04
- **Descrição técnica:** O evento de domínio é gravado na tabela outbox_events dentro da mesma transação @Transactional da operação de negócio; a publicação no RabbitMQ ocorre exclusivamente no callback afterCommit(). Se a instância falha antes do commit, o evento nunca é persistido — consistente. Se falha após o commit, o evento fica retido e é publicado pelo próximo ciclo do scheduler. Essa construção elimina por design a possibilidade de perda silenciosa de eventos, validada nos testes de stress com 600 VUs de escrita simultânea sem nenhum evento perdido.

NR-02: FanoutExchange garante entrega de chat sem perda em ambiente multi-instância

- **Cenários vinculados:** C-03
- **Descrição técnica:** O CommunityBroadcastConsumer replica cada mensagem para todas as instâncias Cloud Run ativas via FanoutExchange, filtrando por INSTANCE_ID para evitar duplicatas. Em todos os perfis de teste (concurrency, load, spike, stress) com até 250 VUs simultâneos, a taxa de entrega foi de 100% (15.145 mensagens enviadas → 294.410 entregues por broadcast), com zero duplicatas — boa decisão para o domínio de chat em ambiente serverless sem coordenação direta entre instâncias.

NR-03: JwtAuthenticationFilter sem vetor de bypass possível

- **Cenários vinculados:** C-05
- **Descrição técnica:** O filtro intercepta toda requisição antes de qualquer controller, validando assinatura, expiração e claims do JWT; apenas rotas explicitamente declaradas em permitAll() escapam da validação. Combinado a @PreAuthorize por papel e rate limiting via Bucket4j antes do pool JDBC, a arquitetura elimina por construção a possibilidade de lógica de negócio ser executada sem autenticação/autorização válida — verificado por análise estrutural do código, sem necessidade de teste de penetração para essa garantia específica.

NR-04: Evolução independente de módulos via arquitetura Hexagonal + eventos

- **Cenários vinculados:** C-06 (crescimento)

- **Descrição técnica:** Como os 11 módulos de domínio só se comunicam por eventos RabbitMQ ou interfaces — nunca por chamada direta —, novas trilhas de recomendação, tipos de notificação ou canais de conteúdo podem ser adicionados sem alterar módulos existentes, reduzindo o risco de regressão em cascata a cada evolução do produto.

5.3 Tradeoffs Técnicos (T)

T-01: Monólito modular vs. microsserviços

- **Cenários vinculados:** C-01, C-02
- **Decisão técnica:** Manter os 11 domínios em um único processo JVM (monólito modular Hexagonal) em vez de extraí-los como microsserviços independentes.
- **Descrição técnica:** A escolha reduz a latência inter-módulo e simplifica o deploy (uma imagem Docker vs. 11 serviços com load balancers e service mesh próprios). A separação por módulo foi projetada para permitir extração futura sem reescrita, já que a comunicação ocorre por interfaces e eventos, não por acoplamento direto entre repositórios. O custo aceito é o escalonamento não granular: quando o motor de recomendação está sob stress, todas as instâncias do monólito sobem juntas, mesmo que os demais módulos estejam ociosos.

T-02: JWT stateless vs. sessions com estado

- **Cenários vinculados:** C-05
- **Decisão técnica:** Autenticação via JWT stateless em vez de sessões centralizadas com estado.
- **Descrição técnica:** O JWT elimina a necessidade de session store e permite escala horizontal sem sticky sessions — qualquer instância Cloud Run valida qualquer token sem consulta centralizada. O custo é que access tokens não podem ser invalidados antes da expiração de 1h; o Refresh Token com rotação mitiga parcialmente o risco ao detectar uso de tokens vazados. O risco residual foi considerado aceitável dado que o Bibloo não processa transações financeiras diretas.

T-03: Fanout-on-Write vs. Fanout-on-Read no feed social

- **Cenários vinculados:** C-01
- **Decisão técnica:** Escrever a publicação na timeline de cada seguidor no momento da publicação (fanout-on-write via RabbitMQ), em vez de calcular o feed dinamicamente na leitura.
- **Descrição técnica:** A escolha torna a leitura do feed $O(1)$ — um simples SELECT no Redis —, mas aumenta proporcionalmente o custo de escrita para publicadores com muitos seguidores. A decisão prioriza a experiência de leitura, que é a

operação mais frequente no produto, em detrimento do custo de escrita, menos frequente.

T-04: Consistência eventual vs. sincronismo via Outbox

- **Cenários vinculados:** C-02, C-04
- **Decisão técnica:** Processar recálculo de recomendações, DNA Literário e fanout de feed de forma assíncrona via consumer, em vez de síncrona dentro da requisição original.
- **Descrição técnica:** O Outbox garante que o evento SERÁ entregue, mas não define QUANDO — a janela de consistência eventual (segundos a minutos) é aceitável para os fluxos assíncronos do Biblío, mas seria inaceitável em domínios com confirmação imediata obrigatória, como pagamentos. A escolha prioriza a velocidade de resposta percebida pelo usuário na escrita, aceitando que o perfil/recomendação atualizados não fiquem visíveis instantaneamente.

5.4 Pontos de Sensibilidade (S)

S-01: Fanout-on-Write no feed social

- **Cenários vinculados:** C-01
- **Componente crítico:** FeedFanoutConsumer / RabbitMQ
- **Descrição técnica:** O fanout escreve na timeline de cada seguidor no momento da publicação. Usuários com um número de seguidores muito acima da média aumentam proporcionalmente a carga de escrita gerada por uma única publicação — uma pequena variação no número de seguidores de um usuário influente pode multiplicar o throughput exigido do consumer de fanout, exigindo atenção caso o produto cresça com perfis de alta audiência.

S-02: Latência do motor de recomendação multi-datastore

- **Cenários vinculados:** C-02
- **Componente crítico:** Camada de orquestração das 6 trilhas de recomendação (Redis + Neo4j + MySQL)
- **Descrição técnica:** O p(95) de 1.210ms sob 400 VUs já é o maior de toda a bateria de testes de desempenho — qualquer degradação latente em qualquer um dos três datastores distribuídos (Neo4j, Redis ou MySQL) se reflete diretamente na latência percebida pelo usuário nas seis trilhas simultaneamente, por serem orquestradas em paralelo na mesma requisição.

S-03: Limite de conexões do CloudAMQP Little Lemur

- **Cenários vinculados:** C-03, C-04
- **Componente crítico:** CloudAMQP Little Lemur (20 conexões simultâneas)

- **Descrição técnica:** Com 10 instâncias Cloud Run, cada uma abrindo de 6 a 8 conexões AMQP, o limite de 20 conexões do plano gratuito pode ser atingido sob escalonamento máximo. Uma variação pequena no número de instâncias simultâneas ativas pode levar o sistema do estado normal direto ao bloqueio de novas conexões, afetando tanto o broadcast de chat quanto a publicação de eventos do Outbox — sem margem gradual de degradação.

6. Plano de Ação e Recomendações Técnicas

As ações abaixo foram organizadas por prazo, priorizando primeiro os riscos com maior probabilidade de materialização sob o volume de usuários atual (R-01, R-05, R-06, R-08) e, em seguida, os riscos cuja origem está em planos gratuitos de infraestrutura ou em decisões estruturais que exigem mudança de padrão arquitetural (R-02, R-03, R-07).

Ações de Curto Prazo (Mitigação Crítica – semanas):

- **Ação 1 – Mitiga R-01:** Calibrar o `maximumPoolSize` do HikariCP em função da soma de threads HTTP ativas e threads de consumer RabbitMQ, e instrumentar métricas de uso do pool via Micrometer/Prometheus com alerta antes da saturação — hoje invisível nos logs de API.
- **Ação 2 – Mitiga R-08:** Implementar monitoramento proativo do número de conexões ativas no CloudAMQP Little Lemur via Prometheus, com alerta ao atingir 15 das 20 conexões disponíveis, e planejar upgrade de plano antes que o escalonamento máximo de instâncias (10×) torne o limite recorrente.
- **Ação 3 – Mitiga R-06:** Introduzir revalidação periódica (heartbeat) da membership do usuário nas conexões WebSocket/STOMP ativas — por exemplo, a cada 60 segundos consultando o status de membership no banco —, fechando a conexão imediatamente em caso de revogação, em vez de depender apenas do handshake inicial.
- **Ação 4 – Mitiga R-07:** Vincular o ciclo de vida do token FCM ao do JWT: invalidar o token de dispositivo em `device_tokens` no momento do logout e da exclusão de conta, eliminando a janela de notificações push para usuários sem sessão ativa.
- **Ação 5 – Mitiga R-05:** Caso o ambiente biblioo-portfolio seja promovido a produção ou receba tráfego crítico, ajustar `min-instances` para um valor maior que zero ou reduzir o intervalo do `@Scheduled` do Outbox, diminuindo a janela de atraso na publicação de eventos pendentes após `scale-to-zero`.

Ações de Médio/Longo Prazo (Sustentabilidade – meses):

- **Ação 6 – Mitiga R-02:** Avaliar a migração do tier Neo4j Aura Free para um plano pago com escalonamento automático antes que o crescimento do grafo social cruze o `threshold` de 5.000ms do RNF-15, ou introduzir cache de subgrafos mais

consultados (co-leitura e similaridade de gosto) no Redis para reduzir a frequência de consultas Cypher pesadas.

- **Ação 7 – Mitiga R-03:** Implementar persistência periódica (snapshot) dos parâmetros α/β do Thompson Sampling no MySQL — por exemplo, a cada N atualizações ou em intervalo fixo —, permitindo recuperação do histórico de aprendizado em caso de flush do Redis, em vez de depender exclusivamente do cache efêmero.
- **Ação 8 – Mitiga R-04:** Introduzir versionamento explícito de contrato na API (ex: prefixo /v1, /v2 ou header de versão), com suporte temporário a múltiplas versões simultâneas no backend e mecanismo de degradação graceful para clientes mobile desatualizados durante a janela de revisão das lojas (1–7 dias).
- **Ação 9:** Implementar testes de caos (Chaos Engineering) simulando indisponibilidade parcial dos datastores distribuídos (Neo4j, Redis, OpenSearch) e do CloudAMQP, validando o comportamento de fallback já existente (ex: fallback FULLTEXT no MySQL) sob condições reais de degradação parcial, e não apenas por análise estrutural do código.
- **Ação 10:** Avaliar uma estratégia de fanout híbrida para o feed social (write para usuários com audiência comum, read para perfis de alta audiência), antecipando o ponto de sensibilidade S-01 antes que o crescimento de seguidores torne o fanout-on-write um gargalo de escrita.

Signed by:

Marcos Alberto Ferreira Pinto

2C40C02291E94D8...

MARCOS ALBERTO
ARQUITETO AVALIADOR

Signed by:

Marcos Alberto Ferreira Pinto

2C40C02291E94D8...

MARCOS ALBERTO
ARQUITETO DO SISTEMA

Signed by:
Carlos José Figueiredo
9BA57DF373BF413...

Assinado por:
Rafael Ganascini de
5751AD815E7B4F3...

Signed by:
Matheus Santos
89D1FAE635F84E9...

Signed by:
Bernardo S. Alvim
AB03A904D888429...

Signed by:
Gabriela Alvarenga
169AFB31B211458...

EQUIPE BIBLIOO
PRODUCT OWNER OU CTO